CineBot

Running the stack locally — a zero-to-chat walkthrough

This guide takes you from an empty machine to a working CineBot installation — installing Docker, configuring your environment, loading the TMDB movie catalogue, and having a real conversation with the bot. No prior Docker or Python experience is assumed.

Total time to complete: **30–60 minutes** (most of it waiting for Docker to build images and download models).

Contents

Part 1 What you need installed

You need **two** things on your computer:

- **Docker Desktop** — runs all the services in containers so you don't have to install Postgres, Redis, Python, etc. individually.
- **A TMDB API key** — free, takes 5 minutes to obtain.

1.1 Install Docker Desktop

Windows

- Go to docker.com/products/docker-desktop/ and download "Docker Desktop for Windows".
- Run the installer — keep the defaults, including "Use WSL 2 instead of Hyper-V" if asked.
- Restart when it asks you to.
- Launch Docker Desktop from the Start menu.
- You'll see a whale icon in the system tray (bottom-right). Wait until it stops animating — that means Docker is ready.

macOS

- Go to docker.com/products/docker-desktop/ and download the version for your chip — "Apple Silicon" for M1/M2/M3/M4, "Intel" for older Macs (check Apple menu → About This Mac).
- Open the .dmg file and drag Docker to Applications.
- Launch Docker from Applications (you may need to approve it in System Settings → Privacy).
- Whale icon appears in the menu bar (top-right). Wait until it says "Docker Desktop is running".

Linux (Ubuntu / Debian)

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker                    # or log out and back in
```

Verify it works

Open a terminal (Windows: PowerShell or Terminal; Mac: Terminal.app; Linux: your usual one) and run:

```
docker --version
docker compose version
```

You should see something like:

```
Docker version 27.3.1, build ce12230
Docker Compose version v2.29.7
```

> *If you get "command not found", Docker Desktop isn't running or the install didn't finish. Close the terminal, re-open Docker Desktop, wait for the whale to stop animating, then open a **new** terminal and try again.*

Give Docker enough RAM

The stack runs 9 containers, including ML models. **Give Docker at least 6 GB of RAM**, 8 GB if you can:

- Docker Desktop → gear icon (top-right) → **Resources** → slide **Memory** to 6 GB or more → **Apply & Restart**.

> *If your machine has less than 8 GB of total RAM, skip the Ollama service later (instructions below) — that alone uses 4–5 GB.*

1.2 Get a TMDB API key

- Go to themoviedb.org/signup and create a free account.
- Check your email and verify it.
- Go to themoviedb.org/settings/api.
- Click **Create** → choose **Developer** → fill in the form. For "Application URL" you can put `http://localhost`; for "Application Summary" write something like "personal movie recommendation project".
- Accept the terms.
- You'll now see an **API Key (v3 auth)** — a long hex string. **Copy it somewhere**; you'll paste it in a minute.

Part 2 Getting the project onto your machine

Download the **cinebot.zip** file attached alongside this guide. It contains the entire repository with the correct folder hierarchy.

- Unzip it somewhere you'll remember.
- Windows: `C:\Users\YourName\cinebot`
- Mac/Linux: `~/cinebot`

2.1 Open a terminal in the project folder

Windows (PowerShell)

```
cd C:\Users\YourName\cinebot
```

Mac/Linux

```
cd ~/cinebot
```

Verify you're in the right place:

```
ls
```

You should see: `README.md`, `docker-compose.yml`, `Makefile`, `backend`, `frontend`, etc. If you don't, you're in the wrong directory.

Part 3 Configuring your environment

3.1 Create the .env file

The project ships with `.env.example` (a template). You need to copy it to `.env` and fill in your TMDB key.

Mac/Linux

```
cp .env.example .env
```

Windows (PowerShell)

```
Copy-Item .env.example .env
```

3.2 Edit .env

Open `.env` in any text editor (VS Code, Notepad, TextEdit — anything). Find this line:

```
TMDB_API_KEY=
```

Paste your TMDB key after the = (no quotes, no spaces):

```
TMDB_API_KEY=abc123def456yourkeyhere
```

Find this line and replace the placeholder with any random string of 16+ characters:

```
JWT_SECRET_KEY=change-me-to-a-long-random-string-in-prod
```

Change to something like:

```
JWT_SECRET_KEY=my-local-dev-secret-for-cinebot-nobody-cares-1234
```

Save and close the file.

3.3 Choose your LLM backend

Look for this line in .env:

```
LLM_PROVIDER=ollama
```

You have three choices:

Option A — Use Ollama (local, free, slower, needs 4+ GB RAM)

- Leave `LLM_PROVIDER=ollama`.
- You'll download a 4 GB model later.

Option B — Use OpenAI (cloud, costs pennies, faster)

- Change to `LLM_PROVIDER=openai`.
- Get an API key at platform.openai.com/api-keys (needs a payment method; GPT-4o-mini costs ~$0.15 per million input tokens — chatting for a week probably costs under $1).
- Find `OPENAI_API_KEY=` and paste your key after the =.

Option C — Skip LLM entirely (simplest for first run)

- The app works fine without it — responses come from templates.

- Leave `LLM_PROVIDER=ollama` and don't pull the Ollama model.
- Template responses are fully readable; the LLM only polishes them.

> ***For your first run, I recommend Option C.*** *You can add the LLM later once everything else works.*

Part 4 Starting the stack

This is the moment of truth. Everything is one command, but it will take **5–15 minutes** the first time because Docker has to download and build a lot of things.

4.1 Start everything

From inside the cinebot folder:

```
docker compose up --build -d
```

What this does, in plain English:

- Downloads images for Postgres, Redis, Grafana, Prometheus, MLflow from the internet.
- Builds the CineBot backend container (installs Python, spaCy, PyTorch, transformers — this is the slow part, ~5 min).
- Builds the frontend container (installs npm packages).
- Starts all 9 services in the background (-d = detached).

What you'll see:

```
[+] Building 234.5s (25/25) FINISHED
 => [api builder 1/5] FROM docker.io/python:3.11-slim-bookworm  ...
 => [api builder 2/5] RUN apt-get update ...
 ...
[+] Running 9/9
 ✔ Container cinebot-postgres     Healthy
 ✔ Container cinebot-redis        Healthy
 ✔ Container cinebot-api          Started
 ✔ Container cinebot-frontend     Started
```

> ***If the build fails with a network error*** *— check that Docker Desktop is running (whale icon) and that you're connected to the internet. Re-run the command.*
> ***If you see "port is already allocated"*** *— something else on your computer is using that port. Most likely Postgres (5432) or Redis (6379). Stop those services or edit* `docker-compose.yml` *to change the ports.*

4.2 Check that everything actually started

```
docker compose ps
```

You should see 9 rows, most showing **Status: running (healthy)**. If any show **Exited** or **Restarting**, check the logs:

```
docker compose logs api --tail 50
```

4.3 Watch logs live (optional but useful first time)

Open a second terminal window, cd into the cinebot folder again, and run:

```
docker compose logs -f api
```

This tails the API logs live. You'll see lines like `app_starting`, `embedder_loading`, `app_ready`. Leave this open while you do the next step — if something breaks, the error shows here.

To stop tailing, press **Ctrl+C** (this doesn't stop the service, just stops watching logs).

Part 5 Initializing the database

The API is running but the database is empty. You need to:

- Create the tables.
- Load the TMDB movie catalogue.
- Generate embeddings for semantic search.

5.1 Apply the schema

```
docker compose exec api alembic upgrade head
```

You should see:

```
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade  -> 20260423_0001, initial schema with pgvector
```

5.2 Load movies from TMDB

```
docker compose exec api python -m app.scripts.ingest_tmdb
```

This takes **3–5 minutes**. You'll see lines like:

```
{"event": "genres_ingested", "count": 19, ...}
{"event": "page_ingested", "page": 1, "count": 20, "total": 20, ...}
{"event": "page_ingested", "page": 2, "count": 20, "total": 40, ...}
...
{"event": "ingest_complete", "total": 1000, ...}
```

You've now got ~1000 popular movies in Postgres.

> *If you see* `tmdb_api_key_missing` *— you forgot to set the key in* `.env`*. Edit* `.env`*, then restart the api (*`docker compose restart api`*) and re-run the ingest command.*

5.3 Generate embeddings

```
docker compose exec api python -m app.scripts.build_embeddings
```

This takes **2–4 minutes** — it runs a small transformer model to convert each movie's description into a 384-dimensional vector. You'll see:

```
{"event": "embedder_loading", "model": "sentence-transformers/all-MiniLM-L6-v2"}
{"event": "embedder_ready", "dim": 384}
{"event": "batch_embedded", "count": 64, "total": 64}
{"event": "batch_embedded", "count": 64, "total": 128}
...
{"event": "embed_complete", "total": 1000}
```

> *The first time this runs, it downloads the model (~90 MB). Don't interrupt it.*

5.4 (Optional) Set up Ollama

Skip this section if you chose Option B (OpenAI) or Option C (no LLM). If you chose Option A (Ollama), download the model:

```
docker compose exec ollama ollama pull llama3.1:8b
```

This downloads **4.7 GB** — grab a coffee. When done:

```
pulling manifest
pulling 8eeb52dfb3bb... 100% ▕▕▕▕▕▕▕ 4.7 GB
verifying sha256 digest
success
```

Part 6 Actually using CineBot

You're ready. Open your browser and go to:

http://localhost:5173

You should see the CineBot auth page — charcoal background, amber accent, a sign-in / register toggle.

6.1 Create an account

- Click **Create** (the right half of the toggle).
- Enter any email — it doesn't need to be real, it's local only: `me@example.com`
- Enter a password of **at least 8 characters**: `letmein1234`
- Click **Create account**.

You'll be dropped into the chat page. The sidebar shows your email and a green "live" dot for the WebSocket connection.

6.2 Have a conversation

Try these in order:

Turn 1 — a recommendation

```
I'm feeling sad, suggest a feel-good movie from the 90s
```

You should see:

- Your message as an amber bubble on the right.
- The assistant's reply with an intro.
- A "TONIGHT'S REEL" header with film-strip sprockets.
- 5 recommendation cards — each with poster, title, year, score, 2–3 reasons, thumbs up/down.

Turn 2 — a refinement

```
more like the second one
```

The recommender averages the embeddings of the first 3 picks and returns a new slate avoiding ones you've already seen.

Turn 3 — give feedback

Click the thumbs-up on any movie. The button turns amber with a checkmark. In the background, this POSTs to `/api/v1/feedback` and queues a Celery task to persist.

Turn 4 — test small talk

```
hey
```

and

```
thanks, bye
```

Both should route to the `greet` and `goodbye` intents respectively.

Turn 5 — test out-of-scope

```
what's the weather tomorrow?
```

You should get the "I'm a movie-only bot" response.

Part 7 Exploring the rest

7.1 API documentation

Open **http://localhost:8000/docs** — an interactive Swagger UI with every endpoint. You can try calls directly from the browser.

To authenticate in Swagger:

- Click the **Authorize** button (top-right).
- You need to first call `/api/v1/auth/login` to get a token.
- Paste the `access_token` from the response into the Authorize dialog.

7.2 Grafana dashboard

Open **http://localhost:3001**

- Username: `admin`
- Password: `admin` (it'll ask you to change it — skip or change, your call).

Left sidebar → **Dashboards** → **CineBot** folder → **CineBot — Service Overview**.

You'll see live traffic, latency percentiles, recommendation counts, LLM stats, and a feedback "like ratio" panel. These only have data once you've chatted a bit.

7.3 Prometheus

http://localhost:9090 — ad-hoc PromQL queries. Try:

```
cinebot_http_requests_total
```

and hit Execute.

7.4 Database inspection

```
docker compose exec postgres psql -U cinebot -d cinebot
```

Inside the psql shell:

```
SELECT COUNT(*) FROM movies;
SELECT title, vote_average FROM movies ORDER BY popularity DESC LIMIT 5;
SELECT role, content FROM messages ORDER BY created_at DESC LIMIT 10;
\q                    -- quit
```

Part 8 Stopping and restarting

Stop everything (keep the data)

```
docker compose down
```

Next time you come back:

```
docker compose up -d
```

No need to rebuild or re-seed — the data is on persistent volumes.

Reset everything (wipe the database, start over)

```
docker compose down -v
```

Then repeat Part 4 onwards.

Part 9 Troubleshooting

"Cannot connect to the Docker daemon"

Docker Desktop isn't running. Open it, wait for the whale icon to stabilise, try again.

Page at localhost:5173 doesn't load

Wait 30 more seconds — the frontend container is the slowest. Then check `docker compose logs frontend`.

"Invalid credentials" on login

You mistyped the email or password, or you haven't registered yet. Use the **Create** tab first.

Chat says "Sorry — ..."

Something in the pipeline failed. Check `docker compose logs api --tail 100` for an error message.

Recommendations are empty

You skipped Part 5.3 (embeddings). Run it.

Build fails with "no space left on device"

Docker is out of disk. In Docker Desktop → Troubleshoot → Clean / Purge data, or reclaim space from old images: `docker system prune -a`.

Everything is slow

Give Docker more RAM (Part 1.1).

Part ∞ Quick-reference summary

Once set up, the whole lifecycle is:

```
cd ~/cinebot                                    # or wherever you put it

docker compose up -d                            # start
docker compose ps                               # check status
docker compose logs -f api                      # watch logs
docker compose down                             # stop
docker compose down -v                          # stop AND wipe data

# inside-container commands
docker compose exec api alembic upgrade head    # apply migrations
docker compose exec api python -m app.scripts.ingest_tmdb
docker compose exec api python -m app.scripts.build_embeddings
docker compose exec postgres psql -U cinebot -d cinebot
```

And the URLs you'll visit:

URL	What
`http://localhost:5173`	The chat UI
`http://localhost:8000/docs`	API Swagger
`http://localhost:3001`	Grafana
`http://localhost:9090`	Prometheus
`http://localhost:5000`	MLflow

When you hit a problem, copy the command you ran and the output you got, and ask Claude — most issues fall into two buckets: (1) Docker not given enough RAM, or (2) skipping one of the init steps in Part 5.

Good luck — and enjoy the movies.